

Build-a-Cart: Step-by-step code (drop-in) — For a beginner

This document shows **exact code** (minimal, friendly) for each step to build the cart system. Copy each file into your project (same folder as `app.py` unless noted). After pasting, restart Flask and test step-by-step.

Note: This is a minimal working example — adjust DB credentials, paths, styles per your project.

0. Install requirements

Run in your project venv:

```
pip install flask mysql-connector-python reportlab
```

1. `app.py` (main Flask app)

Create `app.py` and paste:

```
from flask import Flask, render_template, request, redirect, url_for,
session, flash, send_file, render_template_string
import os
from cart_manager import CartManager
from show import show_items
from invoice_generator import generate_invoice_pdf

app = Flask(__name__)
app.secret_key = 'change_this_to_random_secret'

DB_CONFIG = {
    'host': 'localhost',
    'user': 'root',
    'password': '123',
    'database': 'inventory'
}

# Helpers

def get_cart():
    return session.get('cart', {})

# Routes
```

```

@app.route('/')
def home():
    return redirect(url_for('showw'))

@app.route('/showw')
def showw():
    shower = show_items(DB_CONFIG)
    data = shower.show()
    # data is dict {headers, data}
    return render_template('show.html', data=data['data'],
headers=data['headers'])

@app.route('/cart/add', methods=['POST'])
def add_to_cart():
    name = request.form.get('product_name')
    qty = int(request.form.get('qty', 1) or 1)

    cm = CartManager(DB_CONFIG)
    prod = cm.get_product_by_name(name)
    if not prod:
        flash('Product not found', 'error')
        return redirect(url_for('showw'))

    cart = get_cart()
    if name in cart:
        cart[name]['qty'] += qty
    else:
        cart[name] = {'price': float(prod['Price']), 'qty': qty, 'category':
prod.get('Category', '')}
    session['cart'] = cart
    flash('Added to cart', 'success')
    return redirect(url_for('showw'))

@app.route('/cart')
def view_cart():
    cart = get_cart()
    total = sum(v['price']*v['qty'] for v in cart.values())
    return render_template('cart.html', cart=cart, total=total)

@app.route('/cart/update', methods=['POST'])
def update_cart():
    name = request.form.get('product_name')
    qty = int(request.form.get('qty', 0) or 0)
    cart = get_cart()
    if qty <= 0:
        cart.pop(name, None)
    else:
        if name in cart:
            cart[name]['qty'] = qty
        session['cart'] = cart
        flash('Cart updated', 'success')

```

```

        return redirect(url_for('view_cart'))

@app.route('/cart/remove', methods=['POST'])
def remove_from_cart():
    name = request.form.get('product_name')
    cart = get_cart()
    cart.pop(name, None)
    session['cart'] = cart
    flash('Removed', 'success')
    return redirect(url_for('view_cart'))

@app.route('/cart/clear', methods=['POST'])
def clear_cart():
    session['cart'] = {}
    flash('Cart cleared', 'success')
    return redirect(url_for('view_cart'))

# Download endpoint for invoices
@app.route('/invoices/download')
def download_invoice():
    from flask import request, abort
    fname = request.args.get('file', '')
    if not fname:
        abort(404)
    fname = os.path.basename(fname)
    path = os.path.join(os.getcwd(), 'invoices', fname)
    if not os.path.exists(path):
        abort(404)
    return send_file(path, as_attachment=True)

# Checkout route: generate PDF, clear cart, auto-download then redirect
@app.route('/cart/checkout', methods=['POST'])
def checkout():
    from flask import abort
    cart = get_cart()
    if not cart:
        flash('Cart empty', 'error')
        return redirect(url_for('view_cart'))

    cm = CartManager(DB_CONFIG)
    success, msg = cm.reduce_stock(cart)
    if not success:
        flash(msg, 'error')
        return redirect(url_for('view_cart'))

    try:
        pdf_path = generate_invoice_pdf(cart, save_dir='invoices',
invoice_meta={'customer_name': 'Admin'})
        if not pdf_path or not os.path.exists(pdf_path):
            raise Exception('PDF not created')
    except Exception as e:

```

```

        session['cart'] = {}
        flash(f'Checkout done but invoice failed: {e}', 'error')
        return redirect(url_for('showw'))

    session['cart'] = {}
    session['last_invoice'] = pdf_path
    fname = os.path.basename(pdf_path)
    download_url = url_for('download_invoice') + '?file=' + fname
    inventory_url = url_for('showw')

    # small page to auto-download then redirect
    html = f"""
    <!doctype html><html><body>
    <p>Checkout successful. Download will start...</p>
    <a id='dl' href='{download_url}'>Download</a>
    <script>document.getElementById('dl').click(); setTimeout(function()
    {{window.location='{inventory_url}'}},1500);</script>
    </body></html>
    """
    flash('Checkout complete', 'success')
    return render_template_string(html)

if __name__ == '__main__':
    app.run(debug=True)

```

2. cart_manager.py (DB helper)

Create `cart_manager.py`:

```

import mysql.connector

class CartManager:
    def __init__(self, db_config):
        self.db_config = db_config

    def get_connection(self):
        return mysql.connector.connect(**self.db_config)

    def get_product_by_name(self, name):
        conn = self.get_connection()
        cur = conn.cursor(dictionary=True)
        cur.execute('SELECT Category, Product_name, Price, Quantity FROM
product_avail WHERE Product_name=%s', (name,))
        row = cur.fetchone()
        cur.close(); conn.close()
        return row

    def reduce_stock(self, cart):

```

```

conn = self.get_connection()
cur = conn.cursor()
try:
    conn.start_transaction()
    for name, item in cart.items():
        qty = int(item['qty'])
        cur.execute('UPDATE product_avail SET Quantity = Quantity -
%s WHERE Product_name = %s AND Quantity >= %s', (qty, name, qty))
        if cur.rowcount == 0:
            raise Exception(f'Insufficient stock for {name}')
    conn.commit()
    return True, 'Stock updated successfully'
except Exception as e:
    conn.rollback()
    return False, str(e)
finally:
    cur.close(); conn.close()

```

3. `show.py` (read inventory)

Create `show.py`:

```

import mysql.connector

class show_items:
    def __init__(self, db_config):
        self.db_config = db_config

    def get_db_connection(self):
        try:
            return mysql.connector.connect(**self.db_config)
        except Exception as e:
            print('DB connect error:', e)
            return None

    def show(self):
        mydb = self.get_db_connection()
        headers = ['Category', 'Product Name', 'Price', 'Stock Quantity']
        if not mydb:
            return {'headers': headers, 'data': []}
        cur = mydb.cursor()
        cur.execute('SELECT Category, Product_name, Price, Quantity FROM
product_avail')
        rows = cur.fetchall()
        cur.close(); mydb.close()
        return {'headers': headers, 'data': rows}

```

4. `invoice_generator.py` (use the fixed version you already have)

Put the `invoice_generator.py` file (the fixed one with DejaVu fallback). Ensure `fonts/DejaVuSans.ttf` exists or it will use `Rs.` instead of `₹`.

5. Templates: `templates/show.html` (simple)

Create folder `templates/` and file `show.html` with this minimal code (you can paste your styled version later):

```
<!doctype html>
<html>
<head><meta charset='utf-8'><title>Show</title></head>
<body>
  <h1>Inventory</h1>
  {% with messages = get_flashed_messages(with_categories=true) %}
    {% if messages %}
      {% for cat,msg in messages %}
        <div>{{ msg }}</div>
      {% endfor %}
    {% endif %}
  {% endwith %}

  <a href='{{ url_for("view_cart") }}'>View Cart</a>
  <table border='1'>
    <tr>
      {% for h in headers %}<th>{{ h }}</th>{% endfor %}<th>Action</th>
    </tr>
    {% for item in data %}
      <tr>
        <td>{{ item[0] }}</td>
        <td>{{ item[1] }}</td>
        <td>{{ item[2] }}</td>
        <td>{{ item[3] }}</td>
        <td>
          <form action='{{ url_for("add_to_cart") }}' method='POST'>
            <input type='hidden' name='product_name' value='{{ item[1] }}'>
            <input type='number' name='qty' value='1' min='1'>
            <button type='submit'>Add</button>
          </form>
        </td>
      </tr>
    {% endfor %}
  </table>
</body>
</html>
```

6. Templates: templates/cart.html (simple)

```
<!doctype html>
<html>
<head><meta charset='utf-8'><title>Cart</title></head>
<body>
  <h1>Cart</h1>
  {% with messages = get_flashed_messages(with_categories=true) %}
    {% if messages %}
      {% for cat,msg in messages %}
        <div>{{ msg }}</div>
      {% endfor %}
    {% endif %}
  {% endwith %}

  <a href='{{ url_for("showw") }}'>Back to Shop</a>
  <table border='1'>
    <tr><th>Name</th><th>Price</th><th>Qty</th><th>Total</th><th>Action</
th></tr>
    {% for name,item in cart.items() %}
      <tr>
        <td>{{ name }}</td>
        <td>{{ item.price }}</td>
        <td>
          <form action='{{ url_for("update_cart") }}' method='POST'>
            <input type='hidden' name='product_name' value='{{ name }}'>
            <input type='number' name='qty' value='{{ item.qty }}' min='0'>
            <button type='submit'>Update</button>
          </form>
        </td>
        <td>{{ '%.2f'|format(item.price * item.qty) }}</td>
        <td>
          <form action='{{ url_for("remove_from_cart") }}' method='POST'>
            <input type='hidden' name='product_name' value='{{ name }}'>
            <button type='submit'>Remove</button>
          </form>
        </td>
      </tr>
    {% endfor %}
  </table>

  <p>Total: ₹{{ '%.2f'|format(total) }}</p>

  <form action='{{ url_for("checkout") }}' method='POST'>
    <button type='submit'>Checkout</button>
  </form>
</body>
</html>
```

7. Quick DB schema (MySQL)

Run this in your MySQL to create the table:

```
CREATE DATABASE IF NOT EXISTS inventory;
USE inventory;

CREATE TABLE IF NOT EXISTS product_avail (
  Category VARCHAR(100),
  Product_name VARCHAR(255) PRIMARY KEY,
  Price DECIMAL(10,2),
  Quantity INT
);

INSERT INTO product_avail (Category, Product_name, Price, Quantity) VALUES
('Electronics', 'Laptop', 1200.00, 15),
('Electronics', 'Keyboard', 99.50, 25),
('Accessories', 'Wireless Mouse', 25.99, 50);
```

8. How to test quickly

1. Start MySQL and create DB + table above.
2. Run `python app.py`.
3. Open `http://127.0.0.1:5000/showw` and add items.
4. Visit `http://127.0.0.1:5000/cart` and Checkout.
5. Check `invoices/` folder for generated PDF.

9. Next steps & hardening (short)

- Use product IDs instead of names for robustness.
- Handle concurrent checkouts with proper DB transactions (we use WHERE Quantity >= %s pattern).
- For multi-user, persist carts in DB or Redis.
- Add authentication if not single-admin.

If you want, I can now: - Add a prettier `show.html` + `cart.html` styled with Tailwind (copy-paste ready), or - Add `orders` / `order_items` DB schema and code to save history.

Tell me which one to add next.